

Reviewer Pack — Minimal Number of Monomial Generators and Circuit Linear Reg...

Entry eee9cb6563a6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 08:49:33 UTC

Minimal Number of Monomial Generators and Circuit Linear Regions

Entry ID: eee9cb6563a6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 08:49:33 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Monomial Theory) **Field B** (complexity object): Boolean Circuit Complexity (Linear Regions)

Statement:

For every Boolean circuit C with n inputs, the minimal number of monomial generators $G(n)$ required to represent the polynomial output of C is linearly correlated with the minimum number of linear regions $L(C)$ in the input space such that $G(n) \leq L(C) + \varepsilon$ for some constant ε independent of n .

Rationale (proposer's reasoning):

Monomial generators provide a combinatorial representation of polynomials, while linear regions capture the simplicity of circuit computations. This conjecture suggests that the complexity of representing a polynomial with monomials is closely related to the complexity of computing its output using simple linear operations in the input space.

Taxonomy category: monomial_theory_linear_regions (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 25d9c7d4eab991f2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the ratio $G(n)/L(C)$ must be within a margin of error $\varepsilon \leq 0.1$ for at least 27 out of 30 seeds, where $G(n)$ is the minimal number of monomial generators and $L(C)$ is the minimum number of linear regions in the input space.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'monomial generators' AND 'Boolean circuit complexity' - 'linear regions' INALPHA 'algebraic combinatorics' - $G(n)$ AND $L(C)$ AND 'circuit linear regions'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(n):
        circuit = []
        for _ in range(n):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(random.randint(2, n))]
            circuit.append((gate_type, inputs))
        return circuit

    def evaluate_circuit(circuit, input_values):
        stack = []
        for gate_type, inputs in reversed(circuit):
            if gate_type == 'AND':
                result = all(input_values[i] for i in inputs)
            elif gate_type == 'OR':
                result = any(input_values[i] for i in inputs)
            stack.append(result)
        return stack[0]

    def count_linear_regions(n, circuit):
        regions = set()
        for input_values in itertools.product([0, 1], repeat=n):
            output = evaluate_circuit(circuit, input_values)
            region = tuple(input_values + (output,))
            regions.add(region)
        return len(regions)

    def count_monomial_generators(n, circuit):
        # Placeholder function for counting monomial generators
```

```

    # This is a dummy implementation and should be replaced with actual logic
    return n

n_values = [5, 10, 15, 20, 30, 40]
G_n_sum = 0
L_C_sum = 0
instances_tested = 0

for n in n_values:
    circuit = generate_boolean_circuit(n)
    input_values = [random.randint(0, 1) for _ in range(n)]
    output = evaluate_circuit(circuit, input_values)
    G_n = count_monomial_generators(n, circuit)
    L_C = count_linear_regions(n, circuit)

    if G_n is None or L_C is None:
        return {
            "metric_name": "G(n)/L(C)",
            "metric_value": float('nan'),
            "instances_tested": instances_tested,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    G_n_sum += G_n
    L_C_sum += L_C
    instances_tested += 1

if instances_tested < 30:
    return {
        "metric_name": "G(n)/L(C)",
        "metric_value": float('nan'),
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

G_n_avg = G_n_sum / instances_tested
L_C_avg = L_C_sum / instances_tested

if abs(G_n_avg - L_C_avg) <= 0.1:
    return {
        "metric_name": "G(n)/L(C)",
        "metric_value": G_n_avg,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "G(n)/L(C)",
        "metric_value": G_n_avg,
        "instances_tested": instances_tested,

```

```

        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": f"G(n)={G_n_avg}, L(C)={L_C_avg}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    G_n_avg = sum(r["metric_value"] for r in results if not math.isnan(r["metric_value"])) / len(results)
    L_C_avg = sum(r["instances_tested"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={G_n_avg} std={L_C_avg} support_fraction={support_fraction}")
    elif any(not math.isnan(r["metric_value"]) and abs(r["metric_value"] - G_n_avg) > 0.1 for r in results):
        first_failing_seed = next(i for i, r in enumerate(results) if not math.isnan(r["metric_value"]) and abs(r["metric_value"] - G_n_avg) > 0.1)
        print(f"RESULT: FALSIFIED counterexample=G(n)={G_n_avg} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, making it impossible to evaluate the conjecture's support conditions. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11339
2	propose	ollama_remote	glm4:latest	0	0	16790
3	preregistration	ollama_remote	glm4:latest	0	0	9753
4	novelty	ollama_remote	glm4:latest	0	0	8956
5	novelty	ollama_remote	glm4:latest	0	0	8798
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15511
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10805
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10094
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12834
10	judge	ollama_remote	glm4:latest	0	0	27840

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132719 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/eee9cb6563a6.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/eee9cb6563a6.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/eee9cb6563a6.tar.gz* (if generated)